

Let's Learn TS Fundamentals

What, Why & When

The Golden Circle

What

What is TypeScript?

TypeScript is a statically typed superset of JavaScript, adding optional static types to JavaScript code.

Why

Why TypeScript?

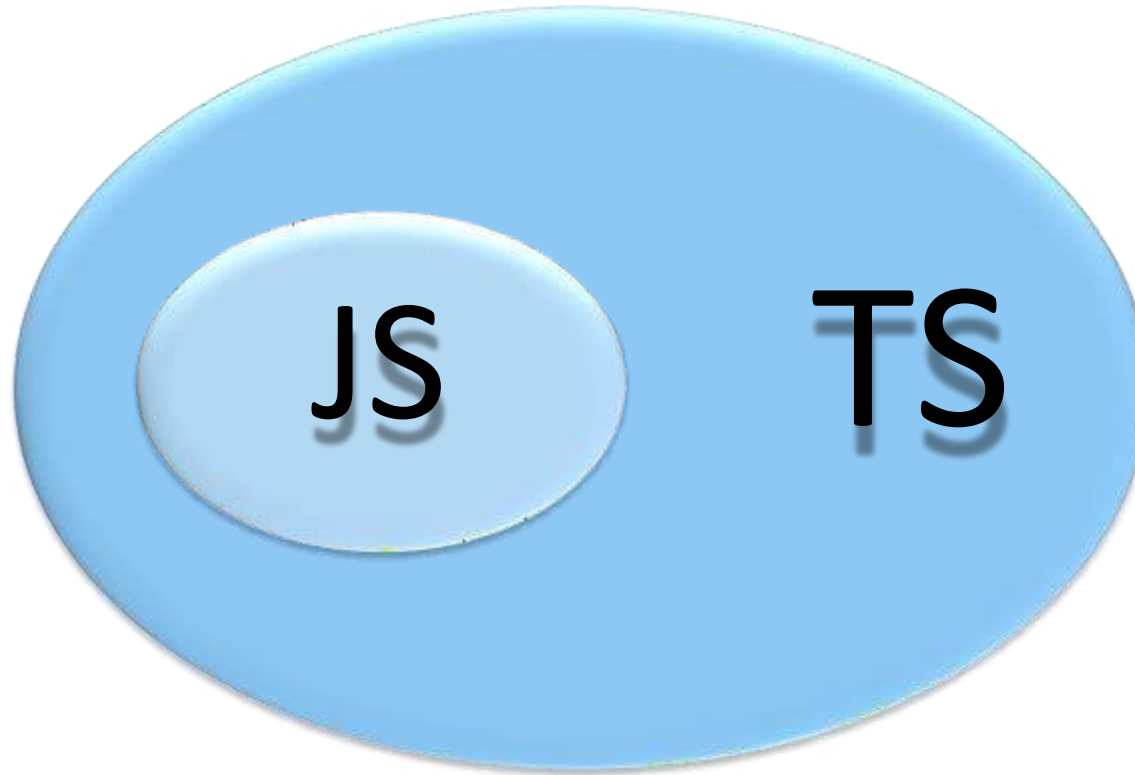
It helps catch errors early, improve code quality, and enhance developer productivity in large-scale projects.

When

When to use TypeScript?

TypeScript is typically used when working on a project where multiple people are collaborating to ensure maintainability and robustness

Introduction to TypeScript



Why prefer TS?

PROS



Has everything JavaScript has, plus additional features



Supports static typing



Discover errors at compile-time

CONS



Takes longer to compile code

Let's add a new TypeScript project

- **Step 1:** Create a new project
- **Step 2:** Open that project in your VS Code (IDE)
- **Step 3:** npm install typescript
- **Step 4:** tsc --version
- **Step 5:** tsc --init

Let's compile and run

- Create a new file demo.ts on your IDE
- Add any JS code (e.g. `console.log("hi, i am TS")`)
- Now run `tsc demo.ts`
- Observe if the demo.js is created
- Now run `node demo.js`

Agenda – 3 (Deep dive into Types)

- Built in types in TS
- Implicit vs Explicit Type Declaration
- Adding custom types
- Combining types

Built in types in TS

JAVASCRIPT

- number
- string
- boolean
- null
- undefined

TYPESCRIPT

- any
- unknown
- tuple

TypeScript - DataTypes :

● any

Definition :

Can hold any type of value (least restrictive).

When to Use:

- Use `any` if you're unsure about the data type or if the variable's type can change dynamically (not recommended for strict type safety).
- Example Scenario: Receiving dynamic API responses.

Example :

```
let response: any;  
response = { name: "Ravi" }; // Object  
response = "Data loaded";  // String
```

TypeScript - DataTypes :

● unknown

Definition :

The `unknown` type is similar to `any`, but stricter.

While `any` bypasses type checks, `unknown` forces to **validate the type** before performing operations on the value.

When to Use:

- Use `unknown` when handling **dynamic data** where the type is uncertain initially but must be validated before use.
- Ideal for working with **user inputs, API responses, or third-party libraries**.
- Example Scenario: Safe Data Handling.

Example :

```
let value : unknown;  
value = 43;  
value="Hello"  
If (typeof value == "string"){  
  Console.log(value.toUpperCase());  
}
```

TypeScript - DataTypes :

● tuple

Definition :

A fixed-length array with **different data types** at specific positions.

When to Use:

- Use tuple when you need a fixed-length array with different types at specific positions.

Example :

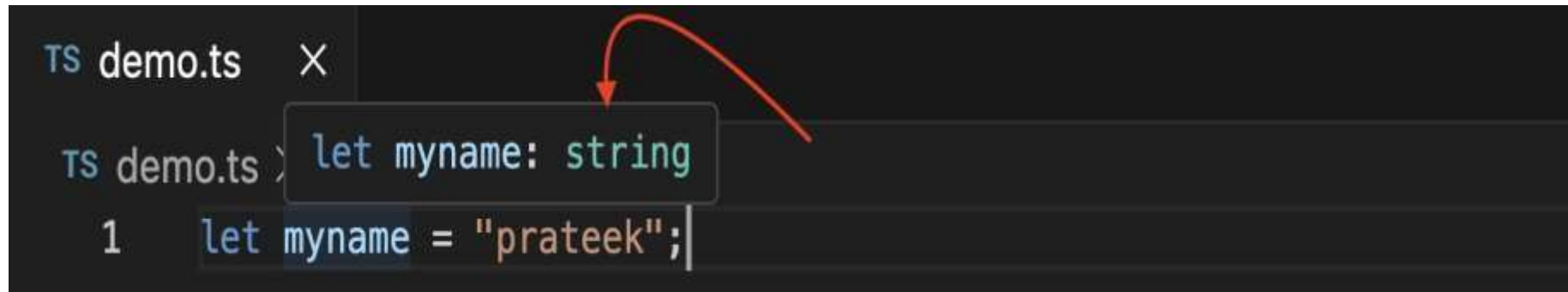
Representing coordinates, database records, etc.

```
let user: [string, number, boolean] = ["Testleaf", 25, true];  
console.log(user[1]); // 25
```

Implicit Types Vs Explicit Types

● Implicit Types :

Means that the type is inferred by TypeScript type inference system which takes responsibility away from us of writing the types



A screenshot of a VS Code editor window titled 'TS demo.ts'. The editor shows two lines of TypeScript code: `let myname: string` on line 1 and `let myname = "prateek";` on line 2. A red arrow points from the `string` type annotation on line 1 to the `myname` variable on line 2, illustrating that the type is inferred from the variable's declaration.

● Explicit Types :

We have to exactly to know and tell what kind of type the value is



A screenshot of a VS Code editor window titled 'TS demo.ts'. The editor shows three lines of TypeScript code: `let myname:string` on line 1, `myname = "prateek"` on line 2, and `console.log(`my name is ${myname}`)` on line 3. A red arrow points from the `string` type annotation on line 1 to the `myname` variable on line 2, illustrating that the type is explicitly declared.

Custom Types in TS

TS demo.ts 1 ●

TS demo.ts > ...

```
1 type testLeafBrowsers = "Chrome"|"Firefox";
```

```
2
```

```
3 let myBrowser:testLeafBrowsers;
```

```
4
```

Type '"Brave"' is not assignable to type
'testLeafBrowsers'. ts(2322)

```
7
```

```
let myBrowser: testLeafBrowsers
```

```
8
```

```
9
```

[View Problem \(⇧F8\)](#) No quick fixes available

```
10 myBrowser = "Brave"
```

```
11
```

Object Literal :

Definition :

In TypeScript (and JavaScript), an object literal is a way to define objects directly using curly braces {}. Access by :

✓ Dot Notation (.)

- Preferred for known property names.

Example

```
const person = { name: "Sam", age: 30 };  
console.log(person.name);    // Output: Sam  
console.log(person.age);     // Output: 30
```

✓ Square Bracket Notation ([])

- Used for dynamic property names with special characters.

Example

```
const car = { "brand-name": "Tesla", model: "Model S", year: 2024 };  
console.log(car["brand-name"]);           // Output: Tesla
```

Type Aliases :

Definition :

A type alias is a way to assign a custom name to a type using the type keyword. It improves code readability, simplifies complex types, and promotes reusability.

✓ Union Type Alias

- Combines multiple types into one.

Example

```
type username = string | number;

let userId: username = "ravi@testleaf.com"; // Valid
userId = 987612345; // Valid
```

✓ Intersection type alias

- Combines multiple types into one with **all properties** included.

Example

```
type Person = { name: string };
type Employee = { employeeId: number };

let manager: Person & Employee = {
  name: "Ravi",
  employeeId: 101
};
```

Type Assertions:

Definition :

Type assertion is a way to tell the TypeScript compiler to treat a value as a specific type. It does **not** change the runtime behavior of the code but helps with type-checking during development.

It is useful when:

- You are sure about the type of a value.
- TypeScript cannot automatically infer the type.

✓ **Syntax:** Two common ways to assert types:

```
//Using angular brackets- <>
let response: any = "Record created successfully";
let myResponse = <string>response;
console.log(myResponse.length);
console.log(typeof myResponse);
```

```
//Using as keyword- as
let sCode: any = "Success";
let statusLength = sCode as string;
console.log(statusLength.length);
```

enum:

Definition :

An **enum** (short for **enumeration**) is a feature in TypeScript that allows you to define a set of **named constants**. It improves code readability, prevents magic numbers/strings, and groups related values together.

✓ Numeric Enums (default)

```
enum Direction {  
  North,    // 0  
  East,     // 1  
  South,    // 2  
  West      // 3  
}  
let dir: Direction = Direction.North;  
console.log(dir); // dir = 0
```

✓ String Enums

```
enum Environment {  
  DEV = "Development",  
  STAGE = "Staging",  
  PROD = "Production"  
}  
let env: Environment = Environment.PROD; // "Production"
```

enum:

✓ String Enums

```
enum Environment {  
  DEV = "Development",  
  STAGE = "Staging",  
  PROD = "Production"  
}  
let env: Environment = Environment.PROD;  
console.log(env); // // "Production"
```

✓ Heterogenous Enums

```
enum Status {  
  SUCCESS = "Success",  
  ERROR = 0  
}  
  
let s1: Status = Status.SUCCESS;  
let s2: Status = Status.ERROR;  
console.log(s1); // "Success"  
console.log(s2); /// 0
```